

Improving accuracy on generating counterfactual questions on reasoning benchmarks by self-verification

Sebastian Orths¹, Frederik Hüttemann²

¹Ruhr-Universität Bochum, Bochum, Germany (sebastian.orth@edu.ruhr-uni-bochum.de)

²Ruhr-Universität Bochum, Bochum, Germany (frederik.huettemann@rub.de)

ABSTRACT

Large language models (LLMs) achieve high accuracy on reasoning benchmarks such as GSM8K, yet this success may not reflect robust causal understanding. We investigate this gap through self-generated counterfactual examples: minimally editing a math word problem so that it yields a specified alternative answer. We introduce a multi-turn solver-verifier harness in which an LLM proposes a counterfactual, critiques it through a separate verification turn, and revises it for up to five iterations. Across eight models on seeded GSM8K subsets, we compare the loop's final output with its initial single-shot candidate using external, post-hoc grading of both target correctness and edit minimality. Single-shot counterfactual validity varies substantially, from 6.0% for Llama-3.1-8B-Instruct to 64.0% for GPT-5.6-Luna, showing that the counterfactual gap is capability-dependent rather than universal. The verifier loop yields substantial, statistically significant improvements for Claude Haiku 4.5 (40.5% to 61.5%) and DeepSeek-V4-Flash (50.0% to 64.3%), but provides little or no benefit for several other models. Our results show that iterative self-verification can improve counterfactual generation when a model has remaining headroom and a verifier aligned with objective validity; self-consistency alone is an unreliable proxy for genuine counterfactual correctness.

Keywords: Artificial Intelligence, Large-Language Models, Counterfactual Examples, Benchmarks, Agentic LLMs, Self-Verification

1. INTRODUCTION

Modern Large Language Models (LLMs) are great at solving reasoning tasks. Most available models are capable of solving reasoning benchmarks. They can even surpass human performance [1]. On grade-school math word problems such as **GSM8K** [2], prompting techniques such as chain-of-thought reasoning and self-consistency sampling [3] have pushed reported accuracies above 90%, which is often taken as evidence that LLMs have learned genuine multi-step arithmetic reasoning rather than surface-level pattern completion.

This apparent mastery is hard to distinguish from very good pattern matching, though. A model that has learned what a typical GSM8K solution looks like can still answer correctly without actually tracking the causal structure of the scenario it is solving. *Counterfactual explanations*, i.e. asking what minimal change to the input would produce a different, specified output, are a long-standing diagnostic in explainable AI for precisely this reason: producing a valid counterfactual requires understanding *why* a model reached its original answer, not just reproducing that answer again. Recent work applies this idea directly to LLMs by asking a model to generate its own *self-generated counterfactual*

examples (SCEs) instead of explaining an externally given edit [4], [5]. The results are sobering. On GSM8K, models that solve the original problems almost perfectly still generate valid single-shot counterfactuals far less reliably, and forcing the edit to be minimal collapses validity even further.

Separately, a growing body of work treats LLMs as agents that can iterate on their own output. Instead of answering once, a model is asked to critique and revise a draft over several turns, sometimes with the help of an independent verifier or an external tool [6], [7], [8]. Verification is central to this idea: process reward models that check every reasoning step instead of only the final answer measurably improve math performance [9], and LLMs are increasingly used to judge other models' outputs automatically [10]. Self-correction is not automatically helpful, though. LLMs that revise their own reasoning without any external feedback have been shown to talk themselves out of correct answers [11], so how a verification loop is actually built matters at least as much as whether one exists at all.

These two research threads, counterfactual probing and agentic self-verification, have mostly been studied separately so far. Existing SCE work stops at single-shot generation, or at best folds a

self-verification instruction into a single generation call, which is not the same as a genuine multi-turn agentic loop with an independent verification step (Section 2 discusses this distinction in more detail). This project asks how the two ideas intersect: when a model is clearly good at a reasoning benchmark like GSM8K but clearly bad at generating single-shot counterfactuals for it, does putting it inside a **solver+verifier agentic loop** recover a measurable part of that gap? We build an agentic harness in which a solver model proposes a counterfactual problem, an independent verifier checks it and gives targeted feedback, and the solver revises its answer accordingly, then measure per model whether looping actually beats a matched single-shot baseline.

2. RELATED WORK

This project builds directly on Dehghanighobadi et al. [4], who are among the first to ask whether LLMs can generate *self-generated counterfactual examples* (SCEs) for their own predictions, instead of merely explaining externally supplied counterfactuals. Across several datasets they find that most instruction-tuned models can produce some valid SCE, but validity depends heavily on the dataset. On **GSM8K**, whose multi-step arithmetic gives an edit many more ways to go wrong than a single-label classification task would, fewer than 20% of generated counterfactuals were valid. That is the sharpest gap among the datasets they test, and it is the starting point of this project.

Mayne et al. [5] extend this line of work by separating two properties that Dehghanighobadi et al. [4] treat together: *validity* (does the edited input actually produce the target output) and *minimality* (is the edit small). Across seven models, including Claude 3.7 Sonnet, GPT-4.1, and o3, and three datasets, they report a robust trade-off. Unconstrained prompting yields close to 100% valid SCEs but mostly non-minimal edits, while explicitly instructing the model to keep the edit minimal collapses validity, even though the edits that do succeed are noticeably smaller. They also test a self-verification-style system prompt that pushes the model to check and revise its own SCE against the constraints before answering. This narrows the gap somewhat, but the check happens inside a single generation call: the model reasons about and revises its own draft within one continuous chain of thought. That does not correspond to agentic behavior in the sense used in this project, since there is no independent verifier, no separate turn boundary, and no possibility of the check disagreeing with a solver it cannot see past.

This style of in-context self-verification sits at the weak end of a broader range of iterative LLM self-improvement methods. **SELF-REFINE** [6] and **REFLEXION** [7] show that letting a model generate feedback on its own output and then revise across several turns improves performance on tasks from code generation to sequential decision-making, while **CRITIC** [8] argues that grounding that feedback in an external tool, such as a code interpreter, rather than the model’s own judgment, is often necessary to actually catch errors. On math specifically, Lightman et al. [9] show that a dedicated process reward model, trained to judge each reasoning step rather than only the final answer, clearly outperforms outcome-only supervision, and LLMs are increasingly deployed as automatic judges of other models’ output more generally [10]. On the other hand, Huang et al. [11] report that LLMs asked to self-correct reasoning without external feedback can end up performing worse, sometimes turning a right answer into a wrong one while “correcting” it. That is exactly the kind of sycophancy risk a same-model, trace-aware verifier is exposed to.

Positioned against this literature, this project is, to our knowledge, among the first to evaluate an agentic solver+verifier loop with a genuinely separate verification turn on the counterfactual-generation gap identified by Dehghanighobadi et al. [4] and sharpened by Mayne et al. [5]. Prior SCE work measures only single-shot generation, and prior self-correction work measures only standard reasoning tasks; this project combines both. The same GSM8K solver+verifier harness is evaluated on the underlying reasoning task as a capability gate, and on counterfactual generation in both single-shot and looped form, so that any recovered validity can be attributed to the agentic loop itself rather than to a stronger base model.

3. METHODOLOGY

3.1. Research Question

The research question is decomposed into three linked claims, each requiring its own measurement.

- **P1**: the model solves GSM8K well (measured by Acc)
- **P2**: its single-shot generation of counterfactuals is “bad” (measured by single-shot Val_{obj})
- **T**: the solver+verifier loop beats single-shot generation (measured by $\Delta Val = Val_{obj}(\text{loop}) - Val_{obj}(\text{single})$, paired on the same items).

As Dehghanighobadi et al. [4] already showed, that

most models perform well on solving GSM8K, the *P1* claim is mostly assumed. It is also investigated whether a solver+verifier loop can beat the single-shot generation for the solving task.

Results of $P2 \leftrightarrow T$ have been evaluated using ΔVal . By using the test proposed by McNemar [12], we evaluate the significance of the observed ΔVal . We assume a *p-value* of less than 0.05 as significant.

3.2. Task Definitions

Two tasks are used. The *solve task* asks the model to solve the original GSM8K question and report its final integer answer. The *counterfactual task* takes the model’s own predicted answer $f(x)$ on the original question and constructs a target $y_{CE} = f(x) + \text{offset}$, where the offset is a seeded, per-item, non-zero integer in $[-10, 10]$. The model is then asked to minimally edit the original problem x so that its correct answer becomes y_{CE} . Targeting the model’s own prediction rather than the dataset’s gold answer follows Dehghanighobadi et al. [4] and keeps the task a genuine self-consistency probe: the model explains a change relative to what it itself believes, not relative to an answer it may never have produced.

3.3. Self-Verification Harness

The harness is built around three roles that can be assigned to the same or to different underlying models. The *solver* produces the original solution and then proposes, and later revises, the counterfactual candidate. The *verifier* is a judge: it is shown the candidate *trace-aware* (additionally shown the solver’s own reasoning) and returns a one-line reason plus a YES/NO verdict. On a NO, the verifier’s reason is fed back to the solver as an additional turn, and the solver revises the candidate; this repeats until the verifier accepts or a hand-settable `max_loops` cap is reached. Grading is decoupled from the verdict: whichever candidate the loop ends on (accepted, or the cap was hit) is independently re-solved by a *checker* to compute a local, secondary validity signal, while the primary validity metric is computed post-hoc by an external grader (Section 3.6).

To yield the single-shot solution, simply the result of the first iteration of the solver is saved. This way, one candidate is generated and graded, and the verifier’s verdict cannot change the outcome. To discover the models abilities we chose that solver and verifier are the same models to evaluate whether a loop makes sense with using one model. Section 5 returns to what that choice does and does not let us conclude. The agentic loop itself was built to be reusable for any

LLM and any self-verification task. Figure 2 in the appendix illustrates the generation process.

3.4. Models

In total four models are used for the *solve task*. In Dehghanighobadi et al. [4], Llama-3.1-8B-Instruct has been used. To stick with similar models, we also choose to test that exact model. Additionally, Qwen2.5-7B-Instruct as an older and Qwen3.5-4B as a more recently released reasoning model were tested. To also use a state-of-the-art, full-size model, Claude-haiku-4.5 has been tested.

For the *counterfactual task*, the model collection has been extended by four, larger models, which were queried through an *OpenRouter API*. This extends the results, as for the *solve task*, all models ran on an RTX 4060 ti with only 16 GB VRAM. The additional models are *mistral-small-2603*, *gemini-3.1-flash-lite*, *gpt-5.6-luna*, and *deepseek-v4-flash* to also represent recently published, full-size models.

3.5. Experimental Conditions

For each of the four solver models, the same seeded 200-item GSM8K subset was run with *single-shot* (`max_loops=5`). Generation uses each model’s own default sampling parameters, rather than a single forced temperature. Solver, verifier, and checker are the same model throughout (self-verifier only); pairing each solver with an independent, stronger verifier is left for future work (Section 5).

The *counterfactual task* was tested using the same four models as solvers, plus the four additional openrouter models. Here, the same 200-item GSM8K subset was run with *loop* (`max_loops=5`) condition. Due to budget limitations for the openrouter models we only used 100-item subsets instead of 200 samples.

3.6. Metrics and Statistical Analysis

For evaluating the accuracy of the model on the original GSM8K problems, the final answer is extracted from the model’s output and compared to the gold answer. The solve accuracy is computed as the proportion of correctly solved problems over the total number of problems attempted. Confidence intervals for solve accuracy are calculated using the Wilson score interval method as only a subset of the GSM8K dataset is used.

$$\text{Accuracy} = \frac{\# \text{Correctly Solved Problems}}{\# \text{Total Problems}} \quad (1)$$

The Wilson score interval is used to compute the 95% confidence intervals for the solve accuracy. [13]

To evaluate the validity of the generated counterfactual examples, we define a candidate as *valid* if it meets two criteria: (1) the model’s counterfactual task can be solved to the target output y_{CE} , and (2) the edit is judged to be minimal. The validity metric is computed as the proportion of valid counterfactuals over the total number of generated candidates.

To automatically grade the results, the newest and most capable Claude Opus 4.8 [14] model is taken. The validation is post-hoc and kept outside the local generation loop. A candidate is *valid* (Val_{obj}) if and only if Opus’ independent re-solve of the candidate matches y_{CE} and Opus judges the edit minimal. The two components (solve rate, minimal rate) and a normalized edit distance are also logged individually as diagnostics.

Additionally, each model resolves the final CE by themselves giving the metric Val . Note, that this metric is not reliable as a source of truth, which is shown and discussed later.

ΔVal_{obj} is computed per model on the items shared by both conditions and tested for significance with an exact McNemar test. [12] Additionally, Wilson 95% confidence intervals are reported for all proportions.

4. RESULTS

This section reports the outcome of the two tasks. In total 12 evaluations are done while four focus on the effect of the agentic harness on the concrete benchmark solving accuracy. Eight models were tested on the hypothesis that the validity of counterfactuals can be increased using a verifier loop. The local models took about 9 days to complete while the tests on bigger models using an API only took a few hours to complete.

4.1. Solve Task

First, the effect of solving the benchmark questions with an agentic harness is investigated. Table 1 reports the accuracy of each model on the original GSM8K problems. Additionally, the accuracy of the self-verification setup with a 95% Wilson confidence interval is given.

The performance of the Qwen model family does not change when using a self-verification loop. For Llama3.1-8B-Instruct, a decrease can be observed while for haiku-4.5 and increase of two samples can be observed.

Table 2 has an in-depth analysis of the effect of the self-verifier loop on the accuracy. The newer Qwen3.5-4B model simply auto-accepts all the incoming results. Therefore, for all 200 samples, the verifier always judged ”yes” and no additional iterations were taken.

In Appendix 11 a false positive example is given. Both solver and verifier did find the correct answer in their thinking process but failed to interpret the question correctly. Because the verifier auto-accepts the solver’s output, the incorrect result is judged as correct. The other example shows a *True Positive Acceptance* where both solver and verifier conclude on the same result and the verdict ”yes” results in a correct answer.

For Llama-3.1-8B-Instruct, the verifier did the complete opposite and enforced in total 105 additional iterations. The verifier has a recall of about 83% of the 1st answer errors. The self-verification loop however fails to correct itself even though the wrong answer was flagged. Additionally, due to the eagerness of the verifier, 22 initially correct results were falsely rejected resulting in dropping correct answers. This explains the accuracy loss for Llama-3.1-8B-Instruct.

For haiku-4.5 an accuracy of above 96% has been observed for the single-shot trial. Three out of seven initially incorrect 1st answers were actually caught by the verifier. Two of them are corrected. The model has a precision of 100% as correct answers were always judged as correct.

Overall, a few observations could be made based on the given results.

- **Capability $\neq$ Self-Verification:** A model reviewing its own trace either agrees or objects imprecisely. This phenomenon is called *syncophancy*. [15]
- **Detection $\neq$ Correction:** Eventhough LLama’s verifier flagged incorrect answers, the solver failed to correct itself.
- **Loop value tracks verifier precision:** Claude’s haiku-4.5 model is the only positive example where the self-verification harness lead to an improvement, while for Llama-3.1-8B-Instruct, the eager verifier performs worse than single-shot solving..

4.2. Single-Shot Counterfactual Validity

Table 3 reports single-shot Val_{obj} per model, together with its two components (solve-to-target rate, minimal rate) and mean normalized edit distance. This is the direct counterpart of the sub-20% GSM8K figure reported by Dehghanighobadi et al. [4].

Table 1. Solve accuracy per model.

Model	N	Solve Acc. (single-shot)	Solve Acc. (Verifier-Harness) [95% CI]	Δ Accuracy
Qwen2.5-7B-Instruct	200	0.920	0.920 [0.870-0.950]	0.000
Qwen3.5-4B	200	0.890	0.890 [0.840-0.930]	0.000
Llama-3.1-8B-Instruct	200	0.850	0.765 [0.700-0.820]	-0.085
claude haiku-4.5	200	0.965	0.975 [0.940-0.990]	0.010

Table 2. Case analysis of the effect of the self-verification loops per model. *recover* means how many times the agentic loop actually detected a 1st-wrong answer and the verifier feedback led to a correct final answer. For the number of *regress* samples, the agentic loop falsely reported a correct 1st answer and the loop’s result then was incorrect. When the 1st answer is wrong *catch / wrong* and *stamp / wrong* are the fractions of how many times the verifier caught or oversaw the incorrect answer.

Model	number of total loops	recover	regress	catch / wrong	stamp / wrong
Qwen2.5-7B-Instruct	9	1	1	4 / 16	12 / 16
Qwen3.5-4B	0	0	0	0 / 22	22 / 22
Llama-3.1-8B-Instruct	105	5	22	24 / 29	5 / 29
claude haiku-4.5	4	2	0	3 / 7	4 / 7

For the three small, locally-run models the finding replicates cleanly. Llama-3.1-8B-Instruct (6.0%), Qwen2.5-7B-Instruct (10.1%) and mistral-small-2603 (11.0%) all land at or below the 20% mark, i.e. they solve GSM8K almost perfectly (Table 1) yet fail to generate a valid single-shot counterfactual on the large majority of items. The reasoning model Qwen3.5-4B sits slightly higher at 27.7% (partial, $n = 47$).

The picture changes markedly for the recent full-size models queried through OpenRouter. Single-shot Val_{obj} spans a full order of magnitude across the model set, from 6.0% (Llama-3.1-8B) to 64.0% (gpt-5.6-luna). deepseek-v4-flash and gemini-3.1-flash-lite reach 50.0%, and claude-haiku-4.5 40.5%. The sub-20% gap identified by Dehghanighobadi et al. [4] is therefore not a universal property of LLMs but tracks model capability: the newest frontier-scale models already clear it single-shot.

The component decomposition locates the bottleneck. For the weak models the binding constraint is *solving to target*, not minimality: Qwen2.5-7B keeps 78.4% of its edits minimal but only 13.1% of them actually re-solve to y_{CE} , and Llama-3.1-8B solves to target on just 8.0% of items. For the stronger models the balance inverts — they hit the target far more often (gpt-5.6-luna 87.0%, deepseek 71.4%) but pay for it with less minimal, more heavily rewritten edits (minimal rate 69–75%).

4.3. Effect of the Verifier Loop

Table 4 reports $\Delta Val = Val_{obj}(\text{loop}) - Val_{obj}(\text{single})$ per model with its exact McNemar p -value, computed on the items shared between the single-shot and loop conditions (both are drawn from the *same* run, so the comparison is fully paired). This table is the direct answer to the research question.

The loop produces a significant objective gain ($p < 0.05$) for four of the eight models: claude-haiku-4.5 (+0.210, $p \approx 3 \times 10^{-11}$), deepseek-v4-flash (+0.143, $p \approx 1.2 \times 10^{-4}$), Qwen3.5-4B (+0.149, $p = 0.016$, partial) and Llama-3.1-8B (+0.040, $p = 0.021$). For the remaining four — gpt-5.6-luna, gemini-3.1-flash-lite, mistral-small-2603 and Qwen2.5-7B — the change is not significant and the confidence interval crosses zero.

Significance alone is misleading, though. Only haiku-4.5 and deepseek show a gain that is both significant *and* large; Llama’s +0.040 is significant but is a marginal lift off a near-zero floor (6% $\rightarrow$ 10%). The two models with a real objective climb share two properties that none of the others combine: single-shot *headroom* (they are not already at ceiling) *and* a verifier whose accept signal tracks objective validity (Table 6). gpt-5.6-luna is the single strongest model single-shot (64.0%) yet gains nothing, because it accepts on the first iteration almost every time and the loop never fires — a ceiling effect, not a failure. gemini-3.1-flash-lite is objectively

Table 3. Single-shot counterfactual validity per model (P2), objective Opus-4.8 grading. Solve = fraction of single-shot candidates whose independent re-solve equals y_{CE} ; Minimal = fraction judged a minimal edit; Val_{obj} requires both. Edit Dist. is the mean normalized edit distance. [†]OpenRouter, $n = 100$. [‡]partial run, $n = 47$.

Model	Val_{obj} (%)	Solve (%)	Minimal (%)	Edit Dist.
deepseek-v4-flash [†]	50.0	71.4	70.4	0.076
gpt-5.6-luna [†]	64.0	87.0	75.0	0.088
claude haiku-4.5	40.5	42.5	94.5	0.039
gemini-3.1-flash-lite [†]	50.0	64.0	69.0	0.154
Qwen3.5-4B [‡]	27.7	48.9	27.7	0.817
mistral-small-2603 [†]	11.0	17.0	56.0	0.068
Qwen2.5-7B-Instruct	10.1	13.1	78.4	0.072
Llama-3.1-8B-Instruct	6.0	8.0	56.5	0.085

flat despite a highly consistent verifier, because that verifier is calibrated to an *inflated* self-consistency signal rather than to objective validity (Section 4, Observations). The weak models simply lack a calibrated verifier and cannot correct what their self-checker cannot detect.

Table 5 decomposes where the loop candidate’s objective validity is lost, and confirms that the weak models fail primarily on *solves-to-target*: even after up to five revision turns, Qwen2.5-7B and Llama-3.1-8B produce edits that re-solve to the intended target on only 22.6% and 17.5% of items respectively.

4.4. Loop-Depth Curve

Figure 1 shows the self-consistency validity achievable if the loop had been stopped at iteration k ($Val@k$), which falls out of the per-iteration logging at no extra generation cost ($k = 1$ is the single-shot baseline; exact values are in Appendix 13). Note that $Val@k$ here is the local *self-consistency* signal (the checker’s own re-solve), which systematically overstates objective validity by roughly $1.5\text{--}1.8\times$ (Section 4, Observations); it is reported to show the *shape* of the loop’s return, not its absolute level.

Three regimes are visible. The models with headroom climb monotonically and roughly double their self-consistency validity over the budget (haiku $0.435 \rightarrow 0.825$, deepseek $0.78 \rightarrow 0.96$, gemini $0.57 \rightarrow 0.85$). The weak models climb from a low base and plateau early (Qwen2.5 $0.09 \rightarrow 0.175$, Llama $0.065 \rightarrow 0.16$). gpt-5.6-luna is flat ($0.94 \rightarrow 0.94$): it accepts on the first iteration on nearly every item, so the loop never runs — the ceiling effect that explains its null ΔVal in Table 4.

4.5. Verifier Behavior

Table 6 reports, per model, the verifier’s accept rate, the agreement between its accept/reject decision and whether the candidate self-consistently validated, and the mean number of iterations taken to accept. A high accept rate paired with low agreement indicates the same-model sycophancy risk raised in Section 2.

Verifier calibration tracks capability even more cleanly than solve accuracy did. haiku-4.5, deepseek, gpt-5.6-luna and gemini all agree with the self-consistency outcome on ≥ 0.97 of items, while the weak models are unreliable: Qwen2.5-7B rubber-stamps (agreement 0.37, accepting roughly twice its actual valid rate) and Llama-3.1-8B’s accept signal is barely better than a coin flip (0.52). One caveat is critical, and is developed in the Observations: this agreement is measured against *self-consistency* validity, which is itself inflated over objective validity. A verifier can therefore be perfectly consistent with a signal that is objectively wrong — exactly gemini’s case (agreement 0.99, objective $\Delta Val \approx 0$).

4.6. Human Audit

As a check on the primary validity metric, a lightly model-stratified sample of 30 loop candidates (10 each from Qwen2.5-7B, Llama-3.1-8B and haiku-4.5) was hand-judged blind to the grader’s verdict. Table 7 reports agreement between the Opus grader and the human labels. The grader agreed with the human judgement on 26 of 30 items (86.7%). All four disagreements were cases where the grader marked a candidate *invalid* that the human accepted — i.e. the objective metric is, if anything, conservative — and none reversed a valid/invalid boundary that would change a headline in Table 4.

Table 4. Effect of the solver+verifier loop on counterfactual validity (T). Val_{obj} single is the iteration-0 candidate; Val_{obj} loop is the final (accepted or capped) candidate of the same run. [†]OpenRouter, $n = 100$, $\max_loops=3$. [‡]partial run, $n = 47$.

Model	Val_{obj} single (%)	Val_{obj} loop (%)	ΔVal [95% CI]	McNemar p
deepseek-v4-flash [†]	50.0	64.3	+0.143 [0.074, 0.212]	1.2×10^{-4}
gpt-5.6-luna [†]	64.0	63.0	-0.010 [-0.030, 0.010]	1.00
claude haiku-4.5	40.5	61.5	+0.210 [0.150, 0.270]	3×10^{-11}
gemini-3.1-flash-lite [†]	50.0	49.0	-0.010 [-0.075, 0.055]	1.00
Qwen3.5-4B [‡]	27.7	42.6	+0.149 [0.028, 0.270]	0.016
mistral-small-2603 [†]	11.0	15.0	+0.040 [-0.007, 0.087]	0.219
Qwen2.5-7B-Instruct	10.1	12.1	+0.020 [-0.014, 0.054]	0.388
Llama-3.1-8B-Instruct	6.0	10.0	+0.040 [0.010, 0.070]	0.021

Table 5. Objective decomposition of the loop (final) candidate, Opus-4.8 grading: fraction re-solving to y_{CE} , fraction judged minimal, and mean normalized edit distance. [†]OpenRouter, $n = 100$.

Model	Solves-to-target (%)	Minimal (%)	Mean Edit Dist.
deepseek-v4-flash [†]	87.8	71.4	0.069
gpt-5.6-luna [†]	86.0	73.0	0.126
claude haiku-4.5	74.5	79.5	0.076
gemini-3.1-flash-lite [†]	82.0	58.0	0.230
mistral-small-2603 [†]	24.0	46.0	0.110
Qwen2.5-7B-Instruct	22.6	59.8	0.129
Llama-3.1-8B-Instruct	17.5	29.5	0.193

4.7. Observations

In this section we note observations that are not directly covered by the preceding results but that qualify how they should be read.

4.7.1. Self-consistency inflates objective validity

Across all seven fully-graded models the local self-consistency signal (the checker’s own re-solve) overstates objective validity by a roughly constant factor of 1.4–1.8 $\times$ (Qwen2.5 1.86 $\times$, Llama 1.85 $\times$, gemini 1.78 $\times$, mistral 1.67 $\times$, gpt 1.51 $\times$, deepseek 1.49 $\times$, haiku 1.37 $\times$). Initially this looked like a weak-model artefact, but with seven models it is clearly universal and only weakly capability-dependent: even the strongest models inflate by about half again. The practical consequence is that the “loop roughly doubles validity” story

one gets from the self-consistency signal is largely an artefact — under objective grading the loop mostly helps the two models (haiku, deepseek) that combine headroom with a calibrated verifier. One should trust Val_{obj} and never the raw accept/self-consistency signal.

4.7.2. Qwen Series

The two Qwen models fail the loop for opposite-looking but related reasons. Qwen2.5-7B is the archetypal rubber-stamp: it accepts 47.5% of candidates but agrees with the self-consistency outcome on only 37% of items, signing off on its own objectively-invalid counterfactuals. Its edits stay minimal (59.8%) but rarely solve to target (22.6%), so the loop has nothing to grip on and ΔVal is a non-significant +0.02. Qwen3.5-4B is more extreme

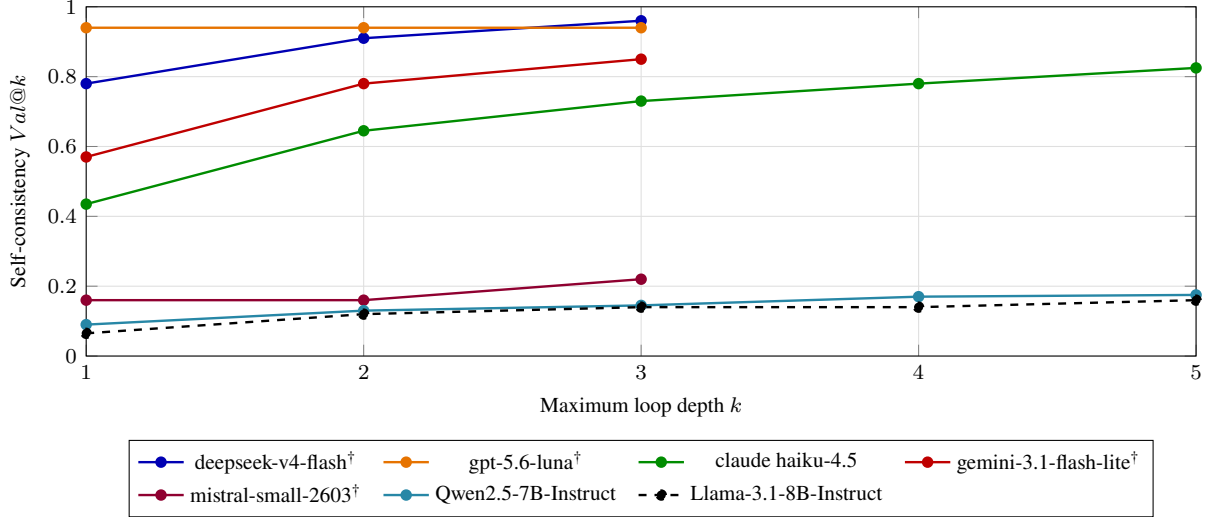


Figure 1. Self-consistency $Val@k$ by maximum loop depth k , per model. OpenRouter models[†] used `max_loops=3`; therefore their lines end at $k = 3$. Values are the local self-consistency signal, not objective Val_{obj} .

Table 6. Verifier accept rate, $\text{accept} \leftrightarrow \text{self-consistency-Val}$ agreement, and mean iterations to accept, loop condition only. [†]OpenRouter, $n = 100$.

Model	Accept Rate	Accept $\leftrightarrow$ Val Agreement	Mean Iters to Accept
deepseek-v4-flash [†]	0.99	0.97	1.25
gpt-5.6-luna [†]	0.96	0.98	1.00
claude haiku-4.5	0.83	0.99	1.87
gemini-3.1-flash-lite [†]	0.86	0.99	1.42
mistral-small-2603 [†]	0.32	0.69	1.69
Qwen2.5-7B-Instruct	0.48	0.37	1.79
Llama-3.1-8B-Instruct	0.31	0.52	2.31

in the solve task, where it auto-accepted all 200 items and never entered a single loop (Section 4, Table 2); on the counterfactual task its partial run does show a significant lift (+0.149, $p = 0.016$), but its enormous mean edit distance (0.817) reveals the mechanism — as a thinking model it tends to rewrite the problem wholesale rather than make the single intended change, so its high solve-to-target comes at the cost of minimality.

4.7.3. GPT-5.6-Luna

gpt-5.6-luna is the clearest counterexample to a naive “stronger models benefit more” reading. It is the single strongest model single-shot ($Val_{obj} = 64.0\%$), its verifier is well calibrated (agreement 0.98), and yet its objective ΔVal is -0.010 (not significant). The reason is a pure ceiling effect: it accepts on the first iteration on nearly every item (mean iterations to accept = 1.00, $Val@1 = Val@3 = 0.94$), so the loop simply never runs. There is no headroom to recover, not a failure to recover it. Capability is thus necessary

but not sufficient for a loop gain; a model already at its single-shot ceiling gets nothing from iterating.

4.7.4. Gemini-3.1-Flash-Lite

gemini-3.1-flash-lite shows that a highly *consistent* verifier is not the same as a correctly *calibrated* one. Its $\text{accept} \leftrightarrow \text{self-consistency}$ agreement is 0.99 — as high as haiku’s — yet its objective ΔVal is -0.010 . The agreement metric scores the verifier against self-consistency Val, and gemini’s self-consistency Val is itself inflated $1.78\times$ over objective. Its tell is edit behaviour: the highest mean edit distance in the set (0.230) and a minimal rate of only 58%. It “solves to target” (82%) by rewriting the problem rather than making the one change the counterfactual asks for, and its verifier happily endorses those large edits.

4.7.5. Llama-3.1

Llama-3.1-8B is the weakest model on both tasks and illustrates the noisy-verifier regime. On the

Table 7. Opus↔human agreement on the audit sample.

N Audited	Opus↔Human Agreement (%)
30	86.7 (26/30)

solve task its eager, low-precision verifier drove a net accuracy *loss* (Section 4); on the counterfactual task its accept signal is barely better than chance (agreement 0.52). Its loop gain (+0.040, $p = 0.021$) is statistically significant but is a marginal lift off a near-zero floor (6% → 10%), and its loop candidates remain both non-solving (17.5% solve-to-target) and non-minimal (29.5% minimal, mean edit distance 0.193). Significance here should not be mistaken for a practically useful improvement.

5. DISCUSSION

The results suggest that iterative solver–verifier loops can improve counterfactual generation, but primarily for models that have sufficient underlying reasoning ability and still have room to improve beyond their single-shot performance. Claude Haiku 4.5 and DeepSeek-V4-Flash show the clearest practically meaningful gains, whereas weaker models often fail to use the verifier’s feedback effectively. At the other extreme, highly capable models such as GPT-5.6-Luna already perform well in the first iteration and therefore gain little from additional turns. The value of an agentic loop consequently depends on both model capability and benchmark saturation: models must be capable enough to revise their answers successfully, but not already so strong that the loop has almost no remaining errors to correct.

Verifier behavior also differs substantially between models. Qwen models tend to over-accept candidates, allowing invalid counterfactuals to pass without revision. Llama-3.1-8B-Instruct exhibits the opposite problem: its verifier frequently rejects candidates, including correct ones, and its solver often fails to produce a successful correction even after an error has been identified. GPT-5.6-Luna occasionally classifies a counterfactual target as impossible even when a valid edit exists. These behaviors show that detecting an error and correcting it are separate capabilities.

An important limitation is the blind verification setup. The verifier evaluates the proposed answer without access to the solver’s full reasoning process. This avoids directly encouraging agreement with the solver’s reasoning, but it also prevents the verifier from identifying the exact step at which an incorrect assumption or calculation was introduced. Providing

access to the reasoning trace could enable more targeted feedback, although it could also increase sycophancy by anchoring the verifier to the solver’s argument. Future work should therefore compare blind, trace-aware, and independently reasoned verification under otherwise identical conditions.

The difference between internal self-consistency and external grading further limits how the loop’s apparent improvement should be interpreted. For example, GPT-5.6-Luna achieves a self-consistency validity of approximately 96%, while its objectively graded validity is only about 63%. Similar inflation is observed across the tested models. Thus, a model and its verifier may agree that a counterfactual is valid even when an independent grader finds that it does not reach the target or is not minimal. Under blind external grading, the improvement produced by the loop is therefore considerably less pronounced than the models’ internal judgments suggest. This finding emphasizes that verifier acceptance should not itself be treated as evidence of correctness.

Several limitations also arise from the experimental design. Counterfactual targets were constructed by adding a random non-zero offset to the model’s original answer. Although this provides a reproducible procedure, some generated targets may be impossible to reach through a genuinely minimal and semantically valid edit. Such items may penalize a model even when its refusal is reasonable. Moreover, GSM8K is a highly saturated benchmark for recent frontier models. Their high baseline performance makes it difficult to observe further improvements and limits the extent to which the findings generalize to more challenging reasoning tasks.

Resource constraints required most experiments to be conducted on relatively small local models and on subsets of GSM8K rather than on the complete test set. The API-based models were evaluated on even smaller subsets, reducing statistical power and making comparisons between model groups less direct. Resource limitations also made fully manual grading infeasible, so a capable LLM was used as the primary external grader. Although the human audit showed substantial agreement and indicated that the automatic grading was generally conservative, model-based evaluation can still introduce systematic errors, particularly when judging edit minimality or

whether a problem is solvable.

6. CONCLUSION

Overall, the experiments provide qualified support for agentic self-verification. Models operating below their single-shot ceiling can benefit from iterative feedback, but the gains are neither universal nor as large as internal self-consistency metrics imply. Effective improvement requires remaining performance headroom, a well-calibrated verifier, and a solver capable of translating criticism into a valid revision. Future studies should evaluate these conditions on less saturated benchmarks, larger and more diverse sample sets, stronger independent verifiers, and counterfactual targets whose feasibility has been established in advance. However of course the improved capability ceiling comes at the cost of increased cost in tokens and increased cost in time.

7. AUTHOR CONTRIBUTIONS

Frederik Hüttemann evaluated the models that were locally run and focused on the implementation of the agentic solving loop and the overall harness. Sebastian Orth evaluated the models that were API based and focused on the counterfactual generation. Both Authors contributed to the paper while Frederik had a greater impact here. Sebastian had a greater impact on the presentation.

8. GENAI USAGE

Architectural and design choices were all human made. Writing content and structure is human made. AI was used to implement code, enhance and rewrite sections of the paper.

9. REFERENCES

- [1] X. Luo et al., “Large language models surpass human experts in predicting neuroscience results,” *Nature Human Behaviour*, vol. 9, no. 2, 305–315, Nov. 2024, ISSN: 2397-3374. DOI: 10.1038/s41562-024-02046-9. [Online]. Available: <http://dx.doi.org/10.1038/s41562-024-02046-9>.
- [2] K. Cobbe et al., *Training verifiers to solve math word problems*, 2021. arXiv: 2110.14168 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2110.14168>.
- [3] X. Wang et al., *Self-consistency improves chain of thought reasoning in language models*, 2022. arXiv: 2203.11171 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2203.11171>.
- [4] Z. Dehghanighobadi, A. Fischer, and M. B. Zafar, *Can llms explain themselves counterfactually?* 2025. arXiv: 2502.18156 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2502.18156>.
- [5] H. Mayne et al., *Llms don’t know their own decision boundaries: The unreliability of self-generated counterfactual explanations*, 2025. arXiv: 2509.09396 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2509.09396>.
- [6] A. Madaan et al., *Self-refine: Iterative refinement with self-feedback*, 2023. arXiv: 2303.17651 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2303.17651>.
- [7] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, *Reflexion: Language agents with verbal reinforcement learning*, 2023. arXiv: 2303.11366 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2303.11366>.
- [8] Z. Gou et al., *Critic: Large language models can self-correct with tool-interactive critiquing*, 2023. arXiv: 2305.11738 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2305.11738>.
- [9] H. Lightman et al., *Let’s verify step by step*, 2023. arXiv: 2305.20050 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2305.20050>.
- [10] L. Zheng et al., *Judging llm-as-a-judge with mt-bench and chatbot arena*, 2023. arXiv: 2306.05685 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2306.05685>.
- [11] J. Huang et al., *Large language models cannot self-correct reasoning yet*, 2023. arXiv: 2310.01798 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2310.01798>.
- [12] Q. McNemar, “Note on the sampling error of the difference between correlated proportions and percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. DOI: 10.1007/BF02295996.
- [13] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [14] Anthropic. “Introducing Claude Opus 4.8.” Product announcement, Accessed: Jul. 14, 2026. [Online]. Available: <https://www.anthropic.com/news/claude-opus-4-8>.
- [15] M. Sharma et al., *Towards understanding sycophancy in language models*, 2025. arXiv: 2310.13548 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2310.13548>.

10. APPENDIX A: PROMPTS

This appendix lists the exact prompt templates used throughout the pipeline: the dataset system prompt, the solver-side counterfactual (CF) revision instruction, the step-checking verifier, the counterfactual verifier (judge), and the post-hoc Haiku grading prompt. Curly braces $\{\dots\}$ denote fields that are filled in programmatically at run time.

10.1. Dataset System Prompt (GSM8K)

Used to frame the original solve for every solver call. $\{\text{PROBLEM}\}$ is substituted with the current question.

You will be given a math problem. The solution to the problem is an integer. Your task is to provide the solution. Only provide the final answer as an integer. Think step by step. Always end your answer with a number. The number should be given after '####'. e.g. if the answer is 78, answer with #### 78. The math problem is $\{\text{PROBLEM}\}$

10.2. Step-Checking Verifier (Solve Task)

A static, dataset-independent system prompt used to check a solver’s step-by-step solution to the original problem. The problem is appended after the prompt text (Problem: $\{\text{question}\}$).

You are a careful math verifier. You are given a problem and another solver’s step-by-step solution. Silently check its reasoning and arithmetic, step by step. Be decisive and terse. Do NOT restate the whole solution and do NOT second-guess yourself in circles. Only judge NO when you are confident the final answer is wrong; if you are unsure, judge YES. End your response with EXACTLY these two lines and nothing after them: Reason: <one short sentence; if wrong, name the first wrong step and the correct value> Verdict: YES or NO

10.3. Counterfactual Revision Instruction (Solver)

Appended as a follow-up user turn onto the solver’s original-solve thread, asking it to minimally revise the problem so its own answer becomes the seeded target $\{\text{target}\}$.

Now, revise the math problem so your final answer to the revised problem becomes $\{\text{target}\}$. Share the revised problem. Besides thinking, only output the revised problem. Start your output with 'Revised problem:'.

Rejection feedback (CF iteration ≥ 2)

If a proposed counterfactual is rejected by the verifier, its one-line Reason is fed back to the solver as follows before it retries:

This CF has a flaw. Its mistake is here: $\{\text{flaw}\}$

10.4. Counterfactual Verifier (Judge)

An independent LLM judge (not the solver) that re-solves the revised problem and checks that it is well-posed and that its answer is exactly the seeded target. $\{\text{question}\}$ is the original problem and $\{\text{target}\}$ is the seeded target integer.

You are a careful math verifier checking a counterfactual question edit. The ORIGINAL problem was: $\{\text{question}\}$

Someone revised it into a new problem, claiming the correct final answer to the revised problem is exactly $\{\text{target}\}$. Silently solve the revised problem yourself, step by step, and check two things:

1. The revised problem is well-posed and natural.
2. Its correct final answer is exactly $\{\text{target}\}$.

Be decisive and terse. Judge NO only when you are confident the revised problem’s answer is NOT $\{\text{target}\}$ (or it is ill-posed); if unsure, judge YES. End your response with EXACTLY these two lines and nothing after them:

Reason: <one short sentence; if NO, name the specific flaw and the answer you actually got> Verdict: YES or NO

Blind judge user turn

In the default (blind) configuration, the judge is shown only the candidate, not the solver’s original reasoning trace, so it re-solves independently without anchoring on the solver’s own solution:

This is the revised math problem. Check whether its correct final answer is exactly $\{\text{target}\}$: $\{\text{candidate}\}$

10.5. Post-hoc Grading Prompt (Claude Opus)

Used offline (not part of the online CF loop) to independently grade whether a finished counterfactual candidate is a *minimal* edit of the original problem, and to cross-check its solved answer. Sent to claude-opus-4.8.

You are grading a counterfactual edit of a math word problem.

ORIGINAL: {question}
 EDITED: {candidate}
 Do two things: (1) solve the EDITED problem step by step and give its final integer answer; (2) decide if EDITED is a MINIMAL edit of ORIGINAL (same scenario, only the change needed to alter the answer). Reply with ONLY a JSON object: {"solved.answer": <int>, "minimal.edit": <true|false>, "reason": "<one line>"}.
 "reason": "<one line>"}.
 "reason": "<one line>"}

11. APPENDIX B: EXAMPLES: SOLVE TASK X QWEN3.5-4B

In the appendix, a few examples of the Qwen3.5-4B reasoning process is given.

11.1. False Positive Acceptance

Question:

Mrs. Tatiana owns a grocery store that sells different fruits and vegetables, which includes carrots. The price of carrots in the grocery store increases by 5% of the original price every year. What would be the price of carrots after three years if it was \$120 initially? (Round to the nearest integer)

Gold answer: 138. The phrase “5% of the original price” fixes the base of the increase: the annual increment is $0.05 \times 120 = \$6$, held constant, so $120 + 3 \times 6 = 138$.

Solver [shortened]:

Variables: $P_0 = \$120, r = 5\% = 0.05, t = 3$.
 The solver flags the ambiguity itself: “increases by 5% of the original price” could mean a fixed \$6 added each year (simple; $\Rightarrow 138$), or a percentage of the current price (compound; $\Rightarrow 138.915$). It resolves this in favor of compound growth on two grounds: (i) yearly price increases “usually compound” in real-world/inflation contexts, and (ii) the instruction to round “implies a non-integer result,” which the simple reading would not produce.

$$P_3 = 120 \times (1.05)^3 = 120 \times 1.157625 = 138.915.$$

Year-by-year: $120 \rightarrow 126 \rightarrow 132.30 \rightarrow 138.915$.
 Rounds to **139**.

139

Verifier [shortened, incorrect]:

The verifier reconstructs both readings and correctly states that “of the original price” is the phrase that *would* select simple interest ($\$6/\text{year} \Rightarrow 138$). It then overrides this literal reading with the same two heuristics the solver used: compound growth as the “standard convention” for price problems, and

the rounding request as a signal that the intended answer is a decimal. On that basis it treats 139 as the intended answer, checks the arithmetic ($120 \times 1.05^3 = 138.915 \rightarrow 139$) as internally consistent, and accepts.

The failure is not arithmetic but interpretive: the verifier identifies the wording that fixes the correct reading, then discards it in favor of a convention heuristic, endorsing an answer that contradicts the literal problem statement.

Verdict: YES (*should be NO*)

11.2. True Positive Acceptance

Question:

Bailey starts with a certain amount of money. Then she receives a weekly allowance of \$5 for 8 weeks. At the end of the 8 weeks, if she has a total of \$100, how much money did Bailey start with?

Gold answer: 60. Starting amount plus $8 \times \$5 = \40 equals \$100, so the start is $100 - 40 = 60$.

Solver [shortened]:

Let x be the starting amount. Total allowance over 8 weeks is $5 \times 8 = \$40$, giving

$$x + 40 = 100 \Rightarrow x = 60.$$

Verifies by accumulating week-by-week from \$60 up to \$100.

60

Verifier [shortened]:

Independently recomputes: $x + (5 \times 8) = 100 \Rightarrow x = 60$, matching the solver at every step. Checks the wording for a hidden condition (e.g. whether the \$100 includes or excludes the allowance) and finds none; the problem is unambiguous. Arithmetic and setup both correct, so it accepts.

Verdict: YES

12. APPENDIX C: COUNTERFACTUAL GENERATION LOOP

The Figure 2 shows the generation process of counterfactual examples.

13. APPENDIX D: LOOP-DEPTH DATA

Table 8 reports the exact values visualized in Figure 1. OpenRouter models used `max_loops=3`, so `Val@4` and `Val@5` are not defined for those models.

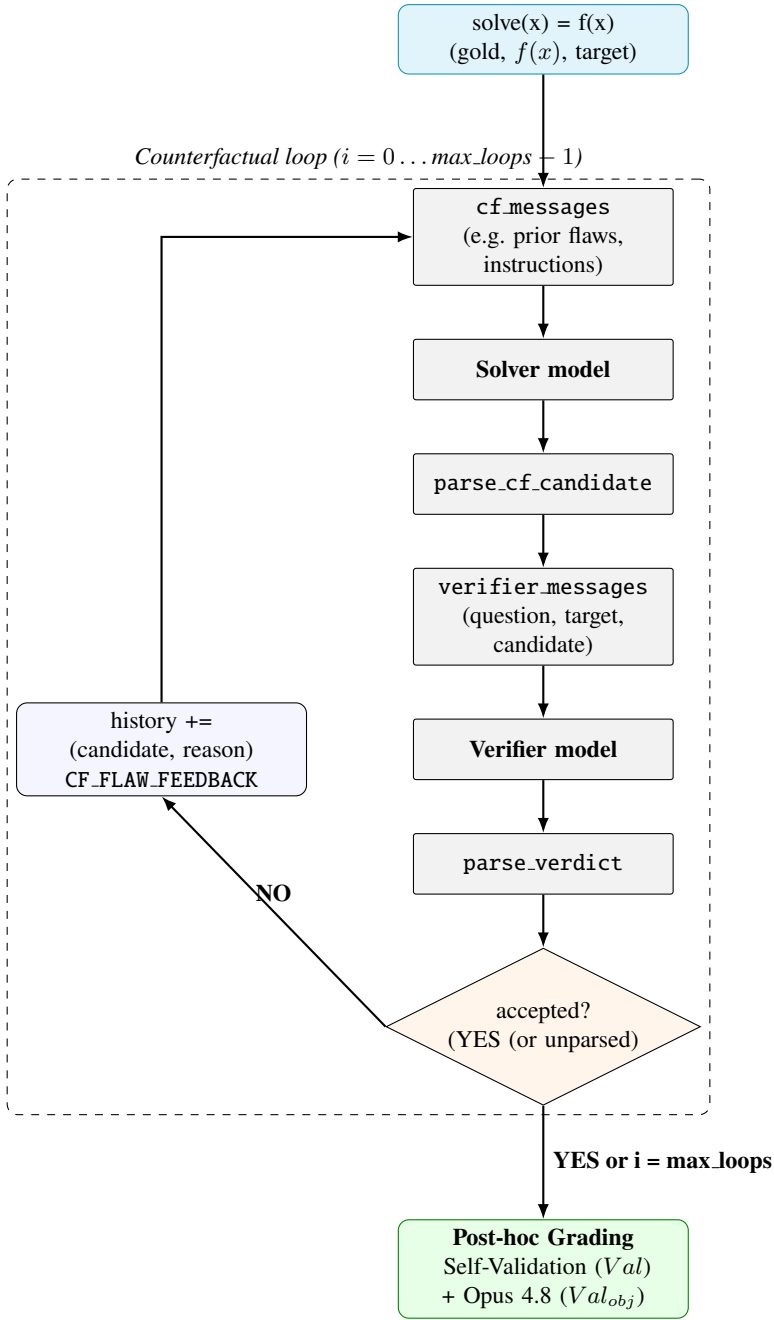


Figure 2. Counterfactual (CF) loop: the solver proposes a candidate, the verifier judges it, and on a NO the reason is fed back to the solver for another revision, until accepted or `max_loops` is reached.

Table 8. Self-consistency $Val@k$ by loop depth k , per model. OpenRouter models[†] used `max_loops=3`, so $Val@4/Val@5$ are not defined. Values are the local self-consistency signal, not objective Val_{obj} .

Model	$Val@1$	$Val@2$	$Val@3$	$Val@4$	$Val@5$
deepseek-v4-flash [†]	0.780	0.910	0.960	—	—
gpt-5.6-luna [†]	0.940	0.940	0.940	—	—
claude haiku-4.5	0.435	0.645	0.730	0.780	0.825
gemini-3.1-flash-lite [†]	0.570	0.780	0.850	—	—
mistral-small-2603 [†]	0.160	0.160	0.220	—	—
Qwen2.5-7B-Instruct	0.090	0.130	0.145	0.170	0.175
Llama-3.1-8B-Instruct	0.065	0.120	0.140	0.140	0.160